

Unit Test Übungen

FAQ: Eigene Exception

Man kann in Java eine eigene Exception erstellen, indem man eine neue Klasse mit einem beliebigen Namen von RuntimeException erben lässt und einen Konstruktor mit String-Parameter hinzufügt:

```
class MyException extends RuntimeException {  
    MyException(String message) {  
        super(message);  
    }  
}
```

Die eigene Exception kann wie normale Java-Exceptions mittels `catch` gefangen und `throw` geworfen werden. Optional kann die Exception zur Methodensignatur hinzugefügt werden (siehe Code in Grau):

```
class ClassToTest {  
    void methodToTest() throws MyException {  
        throw new MyException("Fehler!");  
    }  
}
```

Im JUnit-Test werden Exceptions (auch Eigene) wie folgt geprüft:

```
assertThrows(MyException.class, () -> classToTest.methodToTest());
```

Übung – ListUtil

Schreibe einen Unit Test mit 100 % Testabdeckung für folgende Klasse.
Teste auch mögliche Edge-Cases.

```
final class ListUtil {  
  
    /**  
     * Create and return a List with the following Strings: "This", "will", "be", "tested", "with", "JUnit".  
     */  
    static List<String> createLinkedList() {  
        return new LinkedList<>(List.of("This", "will", "be", "tested", "with", "JUnit"));  
    }  
  
    /**  
     * Returns the most-often-occurring item in the given list.  
     * If there is more than one most-often element, the lowest one is returned.  
     * Tip: Ignore the actual implementation of this method. Instead, focus on the input parameter and the return value for your unit test.  
     * Example: input [4, 4, 3, 2, 1] returns 4  
     * Example: input [4, 3, 3, 2, 2, 1] returns 2  
     * Example: input [4, 3, 2, 1] returns 1  
     */  
    static Integer getMostFrequentElement(List<Integer> integerList) {  
        Map<Integer, Long> elementToFrequency = integerList.stream()  
            .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));  
        return new TreeMap<>(elementToFrequency).entrySet().stream()  
            .max(Map.Entry.comparingByValue())  
            .orElseThrow(() -> new NoSuchElementException("Provided List was empty!"))  
            .getKey();  
    }  
}
```

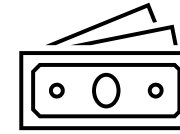
Übung – PayDude

Schreibe einen Unit Test mit 100 % Testabdeckung für folgende App. Teste auch mögliche Edge-Cases.

```
class PayDude {  
  
    private long balance = 0; // Steht für eine beliebige Währung in Cents  
  
    void deposit(long amount) { // Geld einzahlen  
        assertNotNegative(amount);  
        balance += amount;  
    }  
  
    void transfer(long amount, PayDude recipient) { // Geld überweisen  
        assertNotNegative(amount);  
        assertBalanceIsSufficient(amount);  
        balance -= amount;  
        recipient.deposit(amount);  
    }  
  
    private void assertBalanceIsSufficient(long amount) { // Prüfen, ob ausreichend Geld vorhanden ist  
        if (balance < amount) {  
            throw new PayDudeException("Go get some money, dude.");  
        }  
    }  
  
    private void assertNotNegative(long amount) {  
        if (amount < 0) {  
            throw new PayDudeException("Negative amounts are not allowed.");  
        }  
    }  
  
    long getBalance() {  
        return balance;  
    }  
}
```

```
class PayDudeException extends RuntimeException {  
  
    public PayDudeException(String message) {  
        super(message);  
    }  
}
```

Übung – CurrencyConverter



Schreibe einen Unit Test mit 100 % Testabdeckung für folgende App. Teste auch mögliche Edge-Cases.

```
class CurrencyConverter {  
  
    private record CurrencyPair(Currency from, Currency to) {  
        public static CurrencyPair of(Currency from, Currency to) {  
            return new CurrencyPair(from, to);  
        }  
    }  
  
    private final Map<CurrencyPair, ConversionRate> conversions = new HashMap<>();  
  
    CurrencyConverter() {  
        initConversionRates();  
    }  
  
    private void initConversionRates() {  
        conversions.put(CurrencyPair.of(Currency.EURO, Currency.DOLLAR), ConversionRate.of(1.06304));  
        conversions.put(CurrencyPair.of(Currency.DOLLAR, Currency.EURO), ConversionRate.of(0.94056));  
    }  
  
    Money convert(Money fromMoney, Currency toCurrency) {  
        return convert(fromMoney.currency(), toCurrency, fromMoney.value());  
    }  
  
    Money convert(Currency fromCurrency, Currency toCurrency, double amount) {  
        if (amount < 0) {  
            throw new NegativeValueException("Negative amount cannot be converted.");  
        }  
  
        var conversionRate = getConversionRate(fromCurrency, toCurrency);  
        var result = amount * conversionRate.value();  
        return new Money(result, toCurrency);  
    }  
  
    ConversionRate getConversionRate(Currency fromCurrency, Currency toCurrency) {  
        var currencyPair = CurrencyPair.of(fromCurrency, toCurrency);  
  
        var currencyPairExists = conversions.containsKey(currencyPair);  
        if (!currencyPairExists) {  
            throw new CurrencyNotSupportedException(currencyPair.toString());  
        }  
  
        return conversions.get(currencyPair);  
    }  
}
```

```
record ConversionRate(double value) {  
  
    static ConversionRate of(double value) {  
        return new ConversionRate(value);  
    }  
}
```

```
enum Currency {  
    EURO, DOLLAR, INVALID  
}
```

```
record Money(double value, Currency currency) {  
}
```

```
class NegativeValueException extends RuntimeException {  
  
    public NegativeValueException(String message) {  
        super(message);  
    }  
}
```

```
class CurrencyNotSupportedException extends RuntimeException {  
  
    public CurrencyNotSupportedException(String message) {  
        super(message);  
    }  
}
```

Übung – Hospital



Schreibe einen Unit Test mit 100 % Testabdeckung für folgende App. Teste auch mögliche Edge-Cases.

```
class Hospital {  
  
    private final int numberOfBeds;  
    private final LinkedList<Patient> patients = new LinkedList<>();  
  
    Hospital(int numberOfBeds) {  
        this.numberOfBeds = numberOfBeds;  
    }  
  
    boolean add(Patient patient) {  
        if (patient == null || noBedsAvailable() || contains(patient)) {  
            return false;  
        }  
        patients.addLast(patient);  
        return true;  
    }  
  
    boolean contains(Patient patient) {  
        return patients.contains(patient);  
    }  
  
    private boolean noBedsAvailable() {  
        return patientCount() >= numberOfBeds;  
    }  
  
    int patientCount() {  
        return patients.size();  
    }  
  
    Patient healNextPatient() {  
        if (hasPatients()) {  
            return patients.removeFirst();  
        }  
        throw new HospitalException("No patients available.");  
    }  
  
    boolean hasPatients() {  
        return !patients.isEmpty();  
    }  
}
```

```
record Patient(String name) {  
}
```

```
class HospitalException extends RuntimeException {  
  
    public HospitalException(String message) {  
        super(message);  
    }  
}
```

Bonusübung: Hospital

- Schreibe eine Methode `longtermPatients(int max)` für die Klasse `Hospital` (siehe vorherige Übung):
 - Die Methode gibt eine Liste mit Patienten zurück, die sich bisher am längsten im Hospital aufhalten (überlege anhand des Codes in `Hospital`, welche Patienten das sein müssen).
 - Die Patienten werden durch die Methode aber nicht geheilt und nicht entfernt.
 - Der Parameter `int max` limitiert die Anzahl der maximal zurückzugegebenen Patienten.
 - Beispiel: `longtermPatients(2)` gibt eine Collection zurück mit den zwei Patienten, die sich bisher am längsten im Hospital aufhalten.
- Schreibe anschließend einen Unit-Test für die neue Methode (ohne Edge Cases).

Warrior

Schreibe eine Klasse **Warrior** mit folgenden **zwei Attributen** und **passendem Konstruktor**:

```
double health;  
Weapon weapon;
```

Der Typ **Weapon** ist folgendes Interface:

```
interface Weapon {  
    double attack(); // Der Rückgabewert ist der Schaden, den die Waffe macht.  
}
```

Füge der Klasse **Warrior** folgende **Methoden hinzu** (du darfst auch weitere Methoden hinzufügen, falls nötig):

```
void attack(Warrior otherWarrior) // this warrior attacks otherWarrior with current weapon.  
double getHealth() // returns health attribute.
```

Implementiere zwei unterschiedliche Weapons: **RocketLauchner** und **Love**. Denke dir irgendwelche Schadenwerte aus.

Schreibe einen **Unit Test** für die **Klasse Warrior** (teste auch mit unterschiedlichen Weapons, also nicht nur Love für alle Testfälle verwenden).

Es wird kein Mockito für diese Aufgabe benötigt.